

Advanced Programming 2025

Model Comparison of Large-Scale Covariance Estimation for Financial Assets

Final Project Report

Ana Lucía Clavijo Martinez
AnaLucia.ClavijoMartinez@unil.ch
Student ID: 25411315

January 3, 2026

Abstract

The estimation of the covariance matrix of asset returns plays a central role in financial economics, particularly in portfolio allocation and risk management. Portfolio theory for portfolio selection proposes the sample covariance matrix as the standard estimator used to compute the dispersion of a combination of assets. However, its constant use has shown that the sample covariance matrix may suffer from a severe estimation error in finite samples, especially in high-dimensional settings. The main objective of the project is to achieve the implementation of the three alternative estimators (Ledoit-Wolf, Graphical Lasso and DCC-GARCH) for the covariance estimation of a high-dimension data set. The best-performing estimator compared to the sample covariance matrix is expected to be found by the accuracy of the estimation by out-of-sample computation for covariance of the validation data set in both static and dynamic metrics. This study demonstrates that while no single estimator is universally superior, the Ledoit-Wolf shrinkage consistently delivers the highest numerical precision, exhibiting the lowest tracking error and the most accurate Frobenius and Spectral distance across the validation period. In contrast, the GLASSO model has shown superiority in the KL divergence metric, being the best model to represent the probability distribution of the dataset. Additionally, GLASSO should be considered as an important complementary diagnostic tool for its ability to identify significant correlations between assets, a useful feature that dense estimators such as Ledoit-Wolf cannot offer.

Keywords: data science, Python, machine learning, covariance estimation, DCC-GARCH estimation, Glasso, Large-Scale Covariance matrices

Contents

1	Introduction	3
2	Literature Review	3
3	Methodology	6
3.1	Data Description	6
3.2	Approach	6
3.3	Implementation	7
3.4	System architecture	8
4	Results	10
4.1	Experimental Setup	10
4.2	Performance Evaluation	11
4.3	Visualizations	12
5	Discussion	14
5.1	Study limitations	14
6	Conclusion and Future Work	14
6.1	Summary	14
6.2	Future Directions	15
	References	16
A	Technical Formulations	17
A.1	Ledoit-Wolf	17
A.2	Graphical Lasso	17
A.3	DCC-GARCH	17
B	Code Repository	18
B.1	Repository Access	18
B.2	Project Structure	18
B.3	Installation instructions	18
B.4	Reproducibility of Results	18

1 Introduction

Today, the entry to financial markets and the use of financial assets have become accessible to the general public. Due to the appearance of new investment platforms, lower initial investment requirements, and the dispense with brokerage services to enter the market; now the process to start investing is much more uncomplicated than before. However, understanding portfolio performance, risk, variance, and asset correlations can be a challenge for the independent investor when dealing with multiple assets.

Fortunately, along with the development of financial markets, so are the tools used to compute and predict its performance. Given the extensive variety of models that exists now, this project aims to compare different types of models used for variance estimation to find which improves robustness of the estimation while reducing estimation error when working with high-dimensionality matrices. First, the conventional covariance matrix is considered as the baseline model of the project, as this is the simplest way to perform a covariance computation. Moreover, to have a wide range of comparison between the different tools available; a static, a dynamic, and a machine learning model have been chosen to be executed in the project. Under these circumstances, the research question the project looks to answer is: *"Which estimation model performs best for large-scale covariance matrices: DCC-GARCH, Ledoit-Wolf Shrinkage or Graphical Lasso (GLASSO)?"*.

The main objective of the project is to achieve the implementation of the three estimators mentioned above for the covariance estimation of the data set. Additionally, the results of these models will be compared between each other and with the results obtained through the baseline. The performance of all estimators are going to be evaluated under static metrics such as Frobenius error, Spectral norm error, KL divergence, and Portfolio tracking error. Dynamic metrics like rolling windows are also performed in parallel to have a complete validation of each model used.

The best-performing estimator compared to the baseline method is expected to be found by the accuracy of the estimation by out-of-sample computation for covariance of the validation data set in both static and dynamic metrics. These metrics are used to perform a ranking between models based on the error (difference in the covariance estimated between the model and the baseline) to identify the model that has fewer errors compared to the reality. Therefore, the models with the least error will be ranked higher and chosen as the best ones to compute variance in large-scale matrices.

To achieve the proposed goal, a literature review has been performed to find which models can be implemented to carry out the project. Furthermore, the methodology used to execute the project is to establish a database composed of relevant stocks for the last five years. This time series is divided into a training dataset (80% of the original database, meaning the first four years) and a validation dataset (the remaining 20% of the database). The training data set is used to compute and train the submitted models. Finally, these trained models are applied to the validation data set and the results are compared with the baseline to identify the one that performed the best. This entire process is described in greater detail throughout the report.

2 Literature Review

The estimation of the covariance matrix of asset returns plays a central role in financial economics, particularly in portfolio allocation and risk management. Portfolio theory for portfolio selection proposes the sample covariance matrix as the standard estimator used to compute the

dispersion of a combination of assets (Markowitz, 1952). Since it is a nonparametric estimator, its simplicity and easier interpretation have turned the covariance matrix into the natural starting point for both theoretical and empirical studies in this field.

However, its constant use has shown that the sample covariance matrix may suffer from a severe estimation error in finite samples, especially in high-dimensional settings. In the case of reduced available time series observations, Merton (1980) demonstrated a severe estimation error arising from limited time series information. When the number of assets increases, the relation between the number of assets and time observations grows and so does the number of covariance parameters, exacerbating these estimation problems and leading to unstable variance estimates (Jobson and Korkie, 1980).

These limitations, which have been present in the studies mentioned above, motivated the search for more refined methodologies for calculating covariance in large dimension portfolios apart from the sample covariance matrix. This project proposes the Ledoit-Wolf Shrinkage Model, the Dynamic Conditional Correlation - Generalized Autoregressive Conditional Heteroskedasticity (DCC-GARCH) model, and the Graphical Lasso (GLASSO) model as possible alternatives to attain a more accurate computation of covariance in large scale settings¹. Nevertheless, the sample covariance matrix is used as a baseline estimator to provide a transparent and interpretable reference point. All alternative covariance estimation techniques are evaluated relative to this benchmark, allowing a clear assessment of the incremental benefits of using a more elaborate structure for the covariance estimation process.

Firstly, the Ledoit-Wolf estimator utilizes a shrinkage-based approach to minimize the mean squared error between the estimated and true covariance matrices. Through the shrinkage of the sample covariance matrix towards the identity matrix, which is executed as the computation of a convex linear combination of these two matrices, it is possible to obtain a well-conditioned matrix and ensure that it is invertible. This feature is particularly useful in high-dimensional settings, when the number of variables is relatively large to the number of observations, which increases estimation error and produces a non-invertible sample covariance matrix. The Ledoit-Wolf shrinkage estimator is considered an improved version of the sample covariance matrix. Its shrinkage procedure avoids arbitrary tuning and makes minimal structural assumptions, remaining as a distribution-free estimator, allowing for a fair comparison with the baseline (Ledoit and Wolf, 2000).

Given that both the baseline and Ledoit-Wolf estimators are considered as static covariance estimators, the Dynamic Conditional Correlation - Generalized Autoregressive Conditional Heteroskedasticity (DCC-GARCH) model is proposed as a dynamic alternative to estimate the covariance matrix in high-dimensional settings. This multivariate model can show the evolution of the correlation of asset returns over time, decomposing the conditional covariance matrix into two parts, applying a GARCH model individually to each asset, and simultaneously computing a conditional correlation matrix between assets. This separation addresses the dimensionality burden inherent in full multivariate volatility models, making DCC-GARCH a parsimonious and effective model suitable for large asset universes (R. Engle, 2002). The inclusion of this model in the project enables to assess whether time-varying models can improve covariance estimation compared to static and shrinkage-based approaches.

Although the DCC-GARCH model is considered parsimonious, its structure remains dense and can take a long time to compute in high-dimensional settings. In this regard, the Graphical Lasso (GLASSO) estimator can serve as a more straightforward alternative to calculate the co-

¹For the full mathematical formulation and model specifications, see Appendix A on page 17.

variance matrix. This estimator introduces sparsity into covariance estimation by imposing an L1 penalty on the precision matrix, converting small partial correlations to zero. By encouraging sparsity inside the covariance matrix, the Graphical Lasso model reduces estimation variance and ensures that the matrix is invertible, which consequently facilitates its use for calculations in high-dimensional settings (Friedman, Hastie, and Tibshirani, 2008). This model is particularly important for large financial asset universes, where several asset pairs may exhibit a weak correlation that only produce noise when other models are applied to estimate variance. Thus, the usage of GLASSO in this project allows to compare different approaches (shrinkage-based, dynamic models, and sparsity) that attempt to improve covariance calculation in high-dimensional settings.

To test the models and identify which one performs best relative to the baseline, static and dynamic validation measures have been proposed. A normalized score based on these results is used as well to summarize this information and rank the models based on their performance. Ledoit and Wolf (2000) proposed the usage of the Frobenius norm, which measures the overall deviation between estimated and realized covariance matrices but computes each distance element-wise, providing a reliable evaluation of approximation accuracy. Since they give certain relevance to the eigenvalue dispersion effect for the correct implementation of their estimator, the use of the spectral norm is also applicable. Given that it captures the maximum distortion induced by the estimator and focuses on identifying the highest dispersion of eigenvalues, shrinkage of these values can improve the resulting covariance matrix. Both of these metrics are useful for validating model performance in covariance estimation due to their understandability and mathematical precision.

Beyond norm-based distances, the Kullback–Leibler (KL) divergence between multivariate Gaussian distributions offers another criterion for comparing covariance matrices (Kullback and Leibler, 1951). By evaluating discrepancies in the probability distribution between the estimation and reality, the KL divergence can compute the overall dispersion, whereas the previous validation methods focused on individual distances. Complementing these statistical measures, portfolio-based metrics, such as tracking error, provide a financially significant evaluation of the proposed models by quantifying the ex-post variance mismatch of portfolios constructed using predicted covariances (R. F. Engle and Colacito, 2006). For this project, an equally-weighted Global Minimum Variance Portfolio is taken as the reference portfolio to compute the tracking error metric for validation of each model.

To evaluate the performance of each model over time, rolling-window validation is employed to evaluate model stability and robustness across a 60 day window span. By continually computing realized covariance matrices over fixed-length rolling windows, this metric allows to validate the consistency of the estimator and its evolution considering the constant change of input. Rolling validation is widely used in financial econometrics, as it avoids reliance on a single period evaluation and assesses the precision of the model on an out-of-sample basis (Andersen et al., 2003).

Given the different validation metrics proposed to evaluate the precision of the covariance estimation, an aggregated scoring framework is adopted to facilitate model comparison. Individual static metric results are normalized using a min–max scaling to ensure comparability across heterogeneous scales, meaning that the lower the result, the better the performance of the model. The consolidated score is computed as an unweighted average of the normalized results of each static metric for each model (Witten, Frank, and Hall, 2011). This normalization and averaging technique avoids imposing subjective weights on any metric while providing a summary of the performance of each model. The score serves to rank in a straightforward manner which model

outperforms the most the proposed baseline.

3 Methodology

3.1 Data Description

The primary data source are historical daily adjusted stock prices extracted from the Yahoo Finance website. A sample composed of all the companies' stock prices that constitute the S&P 500 index for the last five years (from November 15th of 2020 to November 14th of 2025) is used to test the different covariance estimators proposed. The aforementioned dataset is downloaded through the "yfinance" Application Programming Interface (API), which allows to download the Open, High, Low, Close, and Adjusted Close price, as well as the Volume of trading of the selected stocks. The principal variables used are the tickers to identify each stock and the Adjusted Close price, given that it factors in the possible dividend payouts or stock splits that may have happened during the selected time range. The periodicity throughout the selected window span is on a daily basis, which means that there are around 1,231 observations for each of the 500 tickers, corresponding to the five years that are taken into consideration in this project.

Although the stock sample has been chosen based on the S&P 500 index due to its relevance in financial markets worldwide, its usage also represents some issues with respect to the quality of the database. Given that it is an index whose composition changes depending on the performance of the companies, the structure of the index may have changed in the course of the selected time frame. This means a potential generation of missing values in several of the tickers and dates utilized. For that reason, the appearance of these potential flaws within the database is taken into account during data collection and cleaning.

To address this potential issue, during the training stage, assets are filtered exclusively based on numerical feasibility criteria, namely the absence of excessive missing values in order to be able to compute returns and estimate covariance matrices without numerical failures. Consequently, the final asset universe is defined as the subset of assets for which all covariance estimators can be successfully estimated on the training sample, which is composed of around 491 tickers. This universe is then fixed and held constant throughout the validation and comparison stage, ensuring that all models are evaluated on an identical set of assets and therefore, preventing any mismatch between the training and validation stages.

3.2 Approach

The raw downloaded database has been through a preparation process in order to be prepared to perform model training and validation. First, the training and validation data sets were separated into two different files, following a standard split of 80% for training and 20% for validation. It is worth mentioning that the type of information used is a sequence of time-series that registers the price of each stock on a daily basis. In this context, the database has to be sorted by ticker before the split is performed, so each time-series is divided independently, with respect to its own chronology (Hyndman and Athanasopoulos, 2018).

Both training and validation data sets are then converted from prices to returns, based on the information given in the Adjusted Closed Price. Return is computed as the rate of change between one date and the previous. However, the return is not used in absolute terms. The results obtained are converted into logarithmic returns because in this manner the distribution of the data is approximated to a normal distribution. With this conversion, the data also acquire the additive properties characteristic of the normal distribution, which is preferable for this kind of project (Tsay, 2017). Log-returns are calculated independently for each asset, under the same

logic as the split of data sets, to preserve temporal integrity and avoid cross-sectional distortions between stocks.

Log-returns are the principal input to train and validate the proposed models. As stated above, a conventional covariance matrix is computed and used as the baseline model of the project to see which of the others outperforms this baseline the most. Of the variety of models available, a static, a dynamic, and a machine learning model have been chosen to have different approaches to compare throughout the execution of the project. For the static model, the Ledoit-Wolf Shrinkage Model is executed, which tries to combine the covariance matrix with the identity matrix to try to reduce the covariance values to zero (Ledoit and Wolf, 2000). With respect to the dynamic model, the Dynamic Conditional Correlation - Generalized Autoregressive Conditional Heteroskedasticity (DCC-GARCH) is utilized to have a time-varying scope inside the project. This model is performed in two steps, the first being to compute a series of univariate GARCH estimates, and the second to compute the correlation estimate based on the results obtained before (R. Engle, 2002). Furthermore, for the machine learning model, a Graphical Lasso (GLASSO) is utilized to compute the covariance matrix. The main feature of this model is the usage of an L1 penalty to induce sparsity within the inverse of the covariance matrix, thus optimizing the computation of the covariance matrix (Friedman, Hastie, and Tibshirani, 2008).

To evaluate the performance of each model used throughout the project, the trained models are applied to the validation set, and those results are subject to static and dynamic validation tests. The static metrics utilized are Frobenius error, Spectral norm error, KL divergence, and Portfolio tracking error. The Frobenius norm measures the overall difference between the baseline matrix and the predicted matrix element-wise, the lower the difference, the better the performance of the model. In a similar path, the spectral norm also computes the difference between the baseline and the model prediction, but it focuses on identifying the highest difference between eigenvalues, since that would be the most important source of distortion for the model. The KL divergence compares the probability distribution of the baseline and the model; therefore, a low divergence between probability distributions indicates a higher effectiveness of the predicted matrix in replicating the risk structure and the probabilistic dispersion of the baseline matrix. The Portfolio tracking error aims to measure economic significance, intending to show how much a Global Minimum Variance Portfolio (GMVP) built on predicted covariance differs ex-post.

With regard to dynamic metrics, a validation process through rolling windows of 60 days is performed in parallel to have a complete validation of each model utilized. The best-performing estimator compared to the baseline method is expected to be found by the accuracy of the estimation by out-of-sample computation for variance of the validation data set in both static and dynamic metrics. To facilitate the comparison of the performance of each model, the static metrics are used to construct a ranking between the models based on the error. Based on the difference in the covariance estimated between the model and the baseline, the ranking is able to identify the model that has fewer errors compared to reality. Therefore, the models with the least error will be ranked higher and chosen as the best ones to compute variance in large-scale matrices. This consolidated performance score was derived by normalized aggregation. Each individual evaluation metric underwent min-max normalization to ensure scale consistency, followed by an unweighted averaging process. Within this framework, a lower aggregate score indicates better model performance.

3.3 Implementation

The project is developed in Python 3.10, following a modular structure to ensure scalability and maintainability of the code. In addition, each module is organized clearly and performs a defined

task that contributes to the project as a whole. The libraries used can be classified into five categories: Numerical Data Computing, Machine Learning, Time Series and Econometrics, Data Visualization, and Web Connectivity for Data Acquisition. For the numerical data computing branch, the corresponding libraries are "numpy", "pandas", and "scipy", which are basic to perform data analysis computations. The "scikit-learn", "statsmodels" and "joblib" are the prevailing ones to execute machine learning models. To execute the econometric model DCC-GARCH, the "multigarch" library is used, given that it has a low memory feature that permits to compute successfully this model for a large-scale sample such as the one used in this project. To perform data visualization of the results, the libraries "matplotlib" and "seaborn" are used. Finally, the packages "yfinance", "requests", and "lxml" are utilized to be able to interact with the website "YahooFinance.com" and download the data set of the project.

3.4 System architecture

```
| covariance_estimation/
|-- data/
|-- results/
|   |-- training
|   |-- validation
|-- src/
|   |-- covariance_estimation/
|   |   |-- __init__.py
|   |   |-- 01_data_download.py
|   |   |-- 02_data_split.py
|   |   |-- 03_returns_calculation.py
|   |   |-- 04_baseline_matrix.py
|   |   |-- 05_graphical_lasso_training.py
|   |   |-- 06_ledoit_wolf_training.py
|   |   |-- 07_dcc_garch_training.py
|   |   |-- 08_baseline_validation.py
|   |   |-- 09_model_validation.py
|-- environment.yml
|-- main.py
|-- PROPOSAL.md
|-- README.md
|-- requirements.txt
|-- setup.py
```

Figure 1: Repository Modular Structure

As illustrated in the previous figure, the repository of the project complies with the modular structure stated before. The "data" folder stores csv files resulting from the data download, split, and returns computations. The "results" folder is subdivided into "training" and "validation" folders. "Training" stores the main parameter results in csv format, the main plots in png format, and the trained model in pickle format for each model in their respective folder. The "validation" folder follows the same structure, except for the pickle file, since it is no longer required to be stored.

The "src" directory hosts the core source code and the central logic of the project. Each module contained within this folder represents a key functional component whose specific responsibilities are detailed below:

- 01_data_download.py: dedicated to performing the export of tickers listed in Wikipedia

of the S&P Index in order to have the most recent composition to work with. The code utilized the extracted tickers to search and download its data from "yfinance" API for the selected time range. It outputs a csv file, stored in the "data" folder, with the historical Open, High, Low, Close, Adjusted Close daily prices and the volume of trading of the selected tickers.

- 02_data_split.py: prepared for the sorting of each ticker so that the chronological split of the data is done independently, following a standard 80/20 proportion for training and validation, respectively.
- 03_returns_calculation.py: constructed to calculate the log-returns of each ticker and save it in a csv file for further usage, respecting the previous sorting to ensure temporal integrity of the data.
- 04_baseline_matrix.py: utilized to compute the conventional covariance matrix to be used as the baseline of the project. It uses the log-returns computed previously as input and outputs the covariance matrix in a csv file, which is stored in the "data" folder. It is important to mention that this covariance matrix was computed using only pairwise-complete observations, to avoid missing values if two stocks were not present in the index at any time during the selected period.
- 05_graphical_lasso_training.py: dedicated to perform a Graphical Lasso with L1 penalty, taking the log-returns as input. Before the execution of the model, the data set is lightly imputed to avoid missing values that can affect the final result. The final data set contains 491 tickers (out of the 500 that were downloaded), so it is considered robust and large enough to be used to perform the model without bias. In the end, the covariance matrix and other parameters obtained in csv format, relevant plots in png format, and the trained model in pickle format are saved in its respective folder within the "training" division of the "results" folder.
- 06_ledoit_wolf_training.py: following the same structure as the GLASSO module, this part determines a covariance matrix through a Ledoit-Wolf estimator and stores the same outputs in the "training" folder. To preserve the integrity of the results, the same 491 tickers used in GLASSO are also employed in this methodology.
- 07_dcc_garch_training.py: customized with the "multigarch" library to be able to compute the DCC-GARCH model, this module utilizes the "low memory" feature of the library to perform the necessary computations under low RAM limitations. As before, the module uses the same tickers and stores the same outputs corresponding to this model for further analysis in the validation module.
- 08_baseline_validation.py: utilized to perform the baseline covariance matrix for the validation time frame. It uses as input the log-returns that were split and stored for this stage, corresponding to the latest 20% of the time series of each ticker. To guaranty comparability with the training results, this module filters the tickers before the computations to ensure the usage of the same tickers as those used during the training stage. In addition, the tickers are ordered in the same order as the training baseline covariance matrix to maintain consistency throughout the analysis. The resulting covariance matrix is saved in a csv file in the "validation" division of the "results" folder.
- 09_model_validation.py: to end the pipeline, the trained model went through a validation process that included static and dynamic validation methodologies. On the one hand, for the static metrics, the distances between predicted covariance and realized covariance have been computed with the Frobenius norm, the Spectral norm, the KL divergence,

and the portfolio tracking error. On the other hand, a rolling-window validation has been performed with a window size of 60 days to validate each model. Finally, a consolidated "score" from the static validation metrics has been developed to facilitate the performance comparison of each model to identify the best performing one. This score follows a normalized aggregation and averaging of each static metric for each model, meaning that the lower the score, the better the overall performance of the model.

The "environment.yml" file serves to enhance the reproducibility of the repository since it is made to recreate a virtual environment and to install all the necessary python dependencies of the project. In this way, an environment suitable for running the code can be easily reproduced on different platforms. To complement this, the file "requirements.txt" also contains similar information in case the project has to be reproduced remotely. The "setup.py" file allows one to install the repository as a package, and the "main.py" file is the one dedicated to ensemble all the functions developed through the project and orchestrate the full pipeline at once. Finally, the "README.md" file contains additional information about the correct usage of the repository, and the "PROPOSAL.md" file corresponds to the initial proposal of this project.

4 Results

This section presents the empirical results obtained from the comparative evaluation of the proposed covariance estimation models. Both quantitative performance metrics and graphical diagnostics are reported in order to assess the precision, stability, and economic relevance of each estimator.

4.1 Experimental Setup

The experimental environment of the project was designed to ensure full reproducibility and methodological transparency. This configuration is detailed below:

- **Hardware specifications:** this project was conducted on a standard workstation equipped with a multi-core CPU, with 2 main processors and 4 logic processors and a 12 GB RAM capacity. GPU acceleration is not used, as all models were estimated using CPU-based numerical optimization.
- **Software versions:** The operating system is Microsoft Windows 10 Home. The implementation was carried out in Python 3.10. The corresponding libraries utilized throughout the project initially are "numpy", "pandas", and "scipy". The "scikit-learn", "statsmodels" and "joblib" are the ones used to execute machine learning models. To execute the econometric model DCC-GARCH, the "multigarch" library is used, given that it has a low memory feature that permits to successfully compute this model for large-scale samples. For visualization of results, libraries "matplotlib" and "seaborn" are used. For data collection, "yfinance", "requests", and "lxml" are utilized. All package versions were fixed using a dedicated environment specification to ensure reproducibility.
- **Hyperparameters:** The following model configurations were used:
 - Baseline estimator: sample covariance matrix computed over the training period.
 - Ledoit-Wolf: shrinkage intensity selected automatically following the Ledoit–Wolf procedure.
 - DCC-GARCH: the univariate GARCH was performed under a (1,1) specification and a horizon forecast of 5 days. It is worth mentioning that, given the high dimension of the dataset, the low-memory feature of the "multigarch" library was used to execute this model.

- Graphical Lasso: estimated using cross-validation with five folds for automatic selection of the regularization parameter α .

For the validation process, considering that the validation set has a duration of one year, meaning around 250 observations, a rolling window size of 60 trading days was employed to compute realized covariance matrices to validate the proposed models.

4.2 Performance Evaluation

After the training stage of the models, made with the 80% of the dataset, three trained models were obtained. In order to identify the best performing compared to the baseline, each trained model went through a validation stage. With the remaining 20% of data the trained models were applied to this new information and its results are evaluated via static and dynamic metrics. As mentioned before, the static metrics utilized in this project are Frobenius norm error, Spectral norm error, KL divergence, and Portfolio tracking error, whose results are exposed below:

Table 1: Consolidated Evaluation Metrics for Covariance Estimators

Model	Frobenius	Spectral	KL	Tracking Error
Graphical Lasso	0.07237	0.06871	2405.39	0.000084
Ledoit-Wolf	0.04044	0.03045	2568.23	0.000024
DCC-GARCH	0.06525	0.05430	2362726.98	0.0003127

Table 1 reports the out-of-sample performance of the proposed covariance estimators with multiple static validation criteria. The Frobenius and spectral norms evaluate global and worst-case matrix deviations, respectively, while the Kullback–Leibler (KL) divergence measures the distributional difference between the estimate and reality. The tracking error metric assesses the economic relevance of covariance estimation errors by applying its evaluation in a portfolio context for a GMVP.

On the one hand, the Ledoit-Wolf estimator yielded a shrinkage coefficient of approximately 0.0119. This low value indicates that the sample covariance matrix possesses a robust underlying structure, requiring minimal corrections towards the structured target to ensure mathematical stability. Consequently, the resulting matrix maintains high fidelity to the empirical data while mitigating the typical noise of high-dimensional financial series without making substantial changes. This result explains why the shrinkage estimator achieves the lowest Frobenius and Spectral norms, indicating the most accurate and stable covariance approximation overall. Moreover, it also presents the lowest tracking error, suggesting superior performance for portfolio management applications. These results confirm that linear shrinkage methods significantly outperform the sample covariance matrix by providing a more stable and well-conditioned estimate in high-dimensional environments.

On the other hand, the Graphical Lasso estimator exhibits slightly better performance in terms of KL divergence compared to Ledoit–Wolf, reflecting its ability to capture the probabilistic structure of the data through sparsity in its precision matrix. This result can be explained by the final hyperparameter of the model, which after training yielded an optimal penalty parameter of $\alpha = 0.00025$. This configuration gave a result of a sparsity ratio of 98.61%, reflecting a highly parsimonious network structure between assets where only 1.39% of the potential conditional relations throughout the dataset were retained as significant. However, this approach has reported higher global and spectral errors, as well as higher tracking error relative to Ledoit–Wolf.

The DCC-GARCH model performs comparatively worse across most validation metrics, performing particularly poorly in terms of KL divergence and tracking error. These results underline the challenges of applying multivariate GARCH models in high-dimensional cross-sectional contexts, where this type of time-varying model may suffer to accurately estimate covariance.

To facilitate the comparison of model's performance across these metrics, a consolidated score was constructed. Through an aggregated normalization and averaging, the previous results were converted into a homogeneous scale that allows to compare each model performance easier. Given the score structure, a lower score means better performance for that model. In this way, the Ledoit-Wolf model can be identified as the one that performs best compared to the baseline. The results of the consolidated score are shown in table 2:

Table 2: Normalized Metrics and Final Performance Score

Model	Frobenius_norm	Spectral_norm	KL_norm	TE_norm	score
Graphical Lasso	1	1	0	0.20804	0.55201
Ledoit-Wolf	0	0	0.00006	0	0.00001725
DCC-GARCH	0.77699	0.62332	0	1	0.85008

In summary, the results suggest that the shrinkage-based Ledoit-Wolf estimator provides the best trade-off between statistical precision, numerical stability, and economic relevance in large-scale covariance estimation compared to the baseline.

4.3 Visualizations

While Table 1 reports aggregated performance metrics and Table 2 displays those results in a score, the following figures provide complementary visual evidence to support these quantitative findings. In the case of figure 2, the set of bar plots complements this analysis by offering a visual comparison across metrics, facilitating the identification of relative strengths and trade-offs between covariance estimators. Lower values indicate better agreement with the realized covariance structure. Ledoit-Wolf consistently achieves lower matrix distance metrics, while GLASSO and DCC-GARCH exhibit higher divergence in Frobenius and Spectral norms, reflecting increased estimation variability in high-dimensional settings. Even though the performance of these two models is similar in these metrics, it can be shown that DCC-GARCH performs considerably worse than GLASSO in KL divergence and Tracking error. However, GLASSO shows an advantage in KL divergence, being the best model to replicate the probability distribution of the data overall.

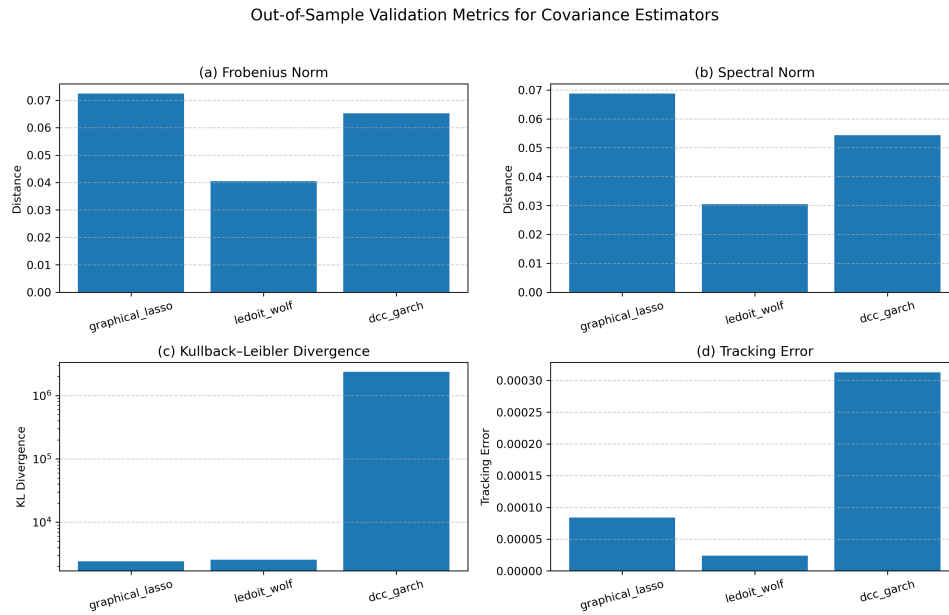


Figure 2: Comparison of static metrics results by model

With respect to dynamic validation metrics of the models, a rolling window validation of 60 trading days was executed. This is shown in figure 3 which illustrates the dynamic stability of the three estimators using a 60-day rolling Frobenius distance. A prominent spike in the distance metric is observed across all models during the mid-year period of 2025, indicating a temporary divergence between the static estimates and the realized market covariance. However, this spike affected the three models equally, meaning that it is due to an external stimulus occurring in the market and is not attributed to deficiencies in the models.

The surge can be explained by the events that took place on April 2, 2025, triggered by the 'Liberation Day' tariff announcement, which produced a turmoil into the U.S. market. This event induced an unprecedented increase in volatility for the S&P 500 index, experiencing a 10.5% draw-down in just 48 hours (SmartTrust, 2025), effectively creating a structural break in the covariance matrix. The sudden shift in asset correlations during this trade-war escalation explains why the static estimates diverged sharply from the realized market behavior. Even the DCC-GARCH model showed difficulty adapting to this uncertainty from the market. The subsequent stabilization of the error metric in late July aligns with the market recovery following the announcement of trade deals and tariff pauses, which restored a degree of predictability to the covariance structure. Despite this external situation, Ledoit-Wolf estimator presented better performance estimating the covariance matrix compared to the other models, having the lowest Frobenius distance during the year in which the rolling window was executed.

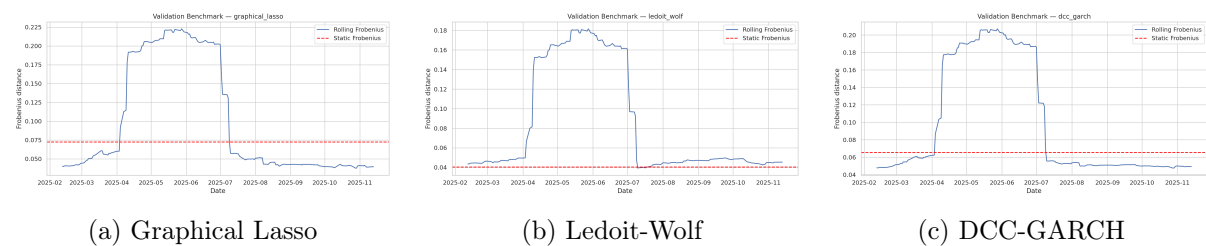


Figure 3: Comparative analysis of covariance estimation models

5 Discussion

The Ledoit-Wolf estimator demonstrated remarkable resilience, consistently providing the most stable and numerically accurate covariance structure throughout the study. Furthermore, the GLASSO successfully achieved its objective of dimensionality reduction, yielding a 98.61% sparsity ratio. This high level of parsimony allowed for the identification of a core dependency network, effectively filtering out systemic noise without losing the fundamental correlations within the asset universe used.

This comparative analysis reveals a clear trade-off between numerical precision and clearer interpretation. While the Ledoit-Wolf estimator achieved slightly lower error scores in terms of Frobenius and Spectral metrics, suggesting a higher degree of mathematical accuracy, the GLASSO provided a far more interpretable and useful representation of the asset universe. By enforcing a 98.61% sparsity ratio, the Graphical Lasso effectively filtered out systemic noise, allowing for the identification of a parsimonious network that contains the 1.39% dependencies that are actually meaningful within the dataset. In this sense, the GLASSO estimator can be viewed as a complementary diagnostic tool rather than the best model to estimate covariance more precisely, since its application might sacrifice numerical precision in exchange for a clearer map of systemic dependencies of assets.

5.1 Study limitations

A key limitation of this study is the reliance on static calibration for the Graphical Lasso and Ledoit-Wolf models. While effective under normal conditions, these estimators suffer to adjust to rapid shifts in real-time. The 60-day rolling window used for validation shows this effect, where extreme shocks, like those seen in April 2025, influence considerably in the error metrics until the volatile period fully exits the estimation window. This issue is particularly important, given that these static models have numerically substantially outperformed the dynamic one (DCC-GARCH), but neither of static nor dynamic models were able to successfully adapt its estimation in a crisis situation.

Another limitation that affects the study's results to a lesser extent is the source of information used, which focuses solely on the U.S. market. By using the S&P index as a reference, the 500 stocks used were part of the same environment, which arises the risk that there may be exogenous risks beyond the U.S. market that may be ignored by the models.

Ultimately, working with high-dimensional matrices such as the one proposed in this project is both a limitation and a challenge in itself. The DCC-GARCH model had serious problems converging at the beginning. Therefore, a specialized python package was required that could run the model without consuming large amounts of memory to execute the model successfully.

6 Conclusion and Future Work

6.1 Summary

In conclusion, this study demonstrates that while no single estimator is universally superior, the Ledoit-Wolf shrinkage consistently delivers the highest numerical precision, exhibiting the lowest tracking error and the most accurate Frobenius and Spectral distance across the validation period. In contrast, the GLASSO model has shown superiority in the KL divergence metric, being the best model to represent the probability distribution of the dataset. Additionally, GLASSO should be considered as an important complementary diagnostic tool for its ability to identify significant correlations between assets, a useful feature that dense estimators such as

Ledoit-Wolf cannot offer.

Finally, the DCC-GARCH model showed a comparatively poor performance in the statics metrics for this high-dimensional setting. Its adaptive capacity is notable during the rolling window validation, but cannot outperform the Ledoit-Wolf performance, which is the most accurate despite the external shock that is witnessed during the validation stage. Therefore, Ledoit-Wolf is chosen as the model that outperforms the baseline the most, having the highest numerical accuracy in estimating covariance in high-dimensional settings. Graphical Lasso can be placed as the second best to outperform the baseline, sacrificing accuracy but giving valuable insights about meaningful asset dependencies. DCC-GARCH is not as accurate as neither of the other models, but under normal conditions it could perform better in making time-varying estimates in high-dimensional settings.

6.2 Future Directions

Regarding future research, exploring hybrid estimation frameworks that combine the mathematical robustness of Ledoit-Wolf with the structural sparsity of the GLASSO to create an ensemble model could be favorable to extract the best features of each model and generate a new one that benefits from these advantages and obtains more accurate and clearer results. Furthermore, the integration of regime-switching models could reduce the effect observed in the DCC-GARCH model during shock events. Such developments could improve the performance of the DCC-GARCH model in response to external stimulus in high-dimensional settings.

Moreover, to expand the scope of the project, a broader dataset can be used that includes different companies across industries and countries to attempt to capture systematic risk worldwide and thus avoid the bias of studying only one market behavior. Also, the dataset can be expanded in the number of observations, considering that a closer number of assets to number of observations ratio could significantly improve the performance of the three models.

References

- Andersen, Torben G. et al. (2003). “Modeling and Forecasting Realized Volatility”. In: *Econometrica* 71.2, pp. 579–625. DOI: 10.1111/1468-0262.00418. URL: <https://doi.org/10.1111/1468-0262.00418>.
- Engle, Robert (2002). “Dynamic conditional correlation: A simple class of multivariate GARCH models”. In: *Journal of Business & Economic Statistics* 20.3, pp. 339–350.
- Engle, Robert F. and Riccardo Colacito (2006). “Testing and Valuing Dynamic Correlations for Asset Allocation”. In: *Journal of Business & Economic Statistics* 24.2, pp. 238–253. DOI: 10.1198/073500106000000030. URL: <https://ssrn.com/abstract=1267010>.
- Friedman, Jerome, Trevor Hastie, and Robert Tibshirani (2008). “Sparse inverse covariance estimation with the graphical lasso”. In: *Biostatistics* 9.3, pp. 432–441.
- Hyndman, Rob J and George Athanasopoulos (2018). *Forecasting: Principles and Practice*. 2nd ed. OTexts. ISBN: 978-0987507112. URL: <https://otexts.com/fpp2/>.
- Jobson, J. D. and B. Korkie (1980). “Estimation for Markowitz Efficient Portfolios”. In: *Journal of the American Statistical Association* 75.371, pp. 544–554. ISSN: 01621459, 1537274X. URL: <http://www.jstor.org/stable/2287643> (visited on 12/29/2025).
- Kullback, Solomon and Richard A. Leibler (1951). “On Information and Sufficiency”. In: *The Annals of Mathematical Statistics* 22.1, pp. 79–86. DOI: 10.1214/aoms/1177729694. URL: <https://doi.org/10.1214/aoms/1177729694>.
- Ledoit, Olivier and Michael Wolf (Nov. 2000). *A well conditioned estimator for large dimensional covariance matrices*. DES - Working Papers. Statistics and Econometrics. WS 10087. Universidad Carlos III de Madrid. Departamento de Estadística. DOI: None. URL: <https://ideas.repec.org/p/cte/wsrepe/10087.html>.
- Markowitz, Harry (1952). “Portfolio Selection”. In: *The Journal of Finance* 7.1, pp. 77–91. ISSN: 00221082, 15406261. URL: <http://www.jstor.org/stable/2975974> (visited on 12/29/2025).
- Merton, Robert C. (Feb. 1980). *On Estimating the Expected Return on the Market: An Exploratory Investigation*. Working Paper w0444. Available at SSRN. National Bureau of Economic Research (NBER). URL: <https://ssrn.com/abstract=262067>.
- SmartTrust (Apr. 2025). *Last Week’s Markets in Review: “Liberation Day” Triggers Market Meltdown*. SmartTrust Weekly Market Review. Accessed: 2026-01-02. URL: <https://www.smarttrustuit.com/wcmu/last-weeks-markets-in-review-liberation-day-triggers-market-meltdown/>.
- Tsay, Ruey S. (2017). *Analysis of Financial Time Series*. 4th ed. Vol. 850. John Wiley & Sons.
- Witten, Ian H., Eibe Frank, and Mark A. Hall (2011). *Data Mining: Practical Machine Learning Tools and Techniques*. 3rd. Burlington, MA: Morgan Kaufmann. ISBN: 978-0-12-374856-0. URL: <https://tjzhifei.github.io/links/DM3.pdf>.

A Technical Formulations

A.1 Ledoit-Wolf

$$\hat{\Sigma}_{LW} = (1 - \delta)\mathbf{S} + \delta\mathbf{F} \quad (1)$$

Where:

- $\hat{\Sigma}_{LW}$ represents the resulting Ledoit-Wolf shrunk covariance matrix
- $\mathbf{S}$ is the sample covariance matrix
- $\mathbf{F}$ is the structured target matrix
- $\delta \in [0, 1]$ is the optimal shrinkage intensity, which determines the weight assigned to the target matrix to minimize the mean squared error

A.2 Graphical Lasso

$$\min_{\Theta \succ 0} \{\text{tr}(\mathbf{S}\Theta) - \log \det(\Theta) + \alpha \|\Theta\|_1\} \quad (2)$$

Where:

- Θ is the precision or inverse covariance matrix, represents conditional dependencies
- $\mathbf{S}$ is the sample covariance matrix
- $\text{tr}(\mathbf{S}\Theta) - \log \det(\Theta)$ represents the negative log-likelihood of the data under a Gaussian assumption
- α is the non-negative regularization parameter that controls the level of sparsity
- $\|\Theta\|_1$ is the ℓ_1 norm, the sum of the absolute values of the off-diagonal elements

A.3 DCC-GARCH

The DCC-GARCH model decomposes the conditional covariance matrix H_t as follows:

$$H_t = D_t R_t D_t \quad (3)$$

Where the dynamic correlation structure is governed by:

$$Q_t = (1 - a - b)\bar{Q} + a(\epsilon_{t-1}\epsilon'_{t-1}) + bQ_{t-1} \quad (4)$$

Where:

- H_t is the time-varying conditional covariance matrix
- D_t is a diagonal matrix of conditional standard deviations obtained from univariate GARCH models
- R_t is the time-varying conditional correlation matrix
- Q_t is the standardized residuals covariance
- $\bar{Q}$ is the unconditional correlation matrix of the standardized residuals
- a and b are non-negative scalar parameters such that $a + b < 1$, representing the impact of past shocks and the persistence of correlation, respectively

B Code Repository

B.1 Repository Access

The complete source code, datasets, and configuration files used in this study are hosted on GitHub at the following URL: <https://github.com/analuuu20/covariance-estimation.git>.

B.2 Project Structure

While the theoretical implementation is discussed in Section 3 (Methodology), the physical organization of the repository is structured as follows:

- `/data`: Raw and processed financial time series in csv files.
- `/src`: Data download and pre processing. Python implementations of the baseline, Ledoit-Wolf, Graphical Lasso, and DCC-GARCH. Execution of validation metrics.
- `/results`: Resulting covariance matrices, validation metrics and plots of both training and validation stages.
- `/main.py`: main file in charge to orchestrate the full pipeline of code in Python.
- `/environment.yml`: virtual environment generated to install the Python dependencies utilized and facilitate project reproducibility.
- `/README.md`: support file for the correct installation and implementation of the repository.

B.3 Installation instructions

To replicate the virtual environment "covariance-estimation", which has all the necessary Python dependencies to run this repository, the source code must be cloned from the repository using a Git-enabled Command Line Interface (CLI), such as Git Bash, Terminal, or PowerShell. The execution of the following commands ensures the integrity of the local directory structure before Python dependencies installation and model implementation:

- to clone the Git repository:

```
git clone https://github.com/analuuu20/covariance-estimation.git
cd covariance-estimation
```

- to install and activate the virtual environment:

```
conda env create -f environment.yml
conda activate covariance-estimation
```

Alternatively, for remote usage the libraries used are in the requirements.txt file. Use:

```
pip install -r requirements.txt
```

For the "multigarch" library, installation is done through Github:

```
pip install git+https://github.com/mechanicpanic/multigarch.git
```

"multigarch" library created by Anna Smirnova.

B.4 Reproducibility of Results

To reproduce the results presented in this document, the virtual environment "covariance-estimation" must be correctly installed. Once activated, the main file can be run to obtain the results shown:

```
python main.py
```